Panadería Alas

Guía del código y del flujo del programa

Manual para el equipo de desarrollo. Versión del 16 de octubre de 2026.

La aplicación sigue usando PHP 8.2, MySQL/MariaDB y JavaScript sin un framework. La reorganización separa el procesamiento de las peticiones, las reglas del negocio y las plantillas. Las URL, los formularios y las hojas de estilo existentes se conservan.

1. **Navegador**
   Solicita una URL o envía un formulario.
2. **Entrada PHP**
   Conserva la dirección que ya usa el proyecto.
3. **Controlador**
   Lee datos, comprueba permisos y prepara la respuesta.
4. **Servicio y MySQL**
   Valida las reglas y guarda la información cuando corresponde.
5. **Vista**
   Muestra los datos como HTML. El navegador aplica CSS y JavaScript.

Cómo leer esta guía

- Para ubicarse: estructura y recorrido de una petición, páginas 2 y 3.
- Para seguir una funcionalidad: usuarios, productos, carrito y pedidos, páginas 4 a 7.
- Para modificar el proyecto: base de datos, rutas y utilidades, páginas 8 a 10.
- Para trabajar y verificar: cambios habituales, pruebas y diagnóstico, páginas 11 a 13.

Las rutas son relativas al repositorio. Las menciones a Core, Controllers, Services y Views corresponden a las carpetas dentro de app. No todos esos archivos se abren en el navegador.

1. Dónde está cada responsabilidad

```
app/
  Core/                Inicio y utilidades comunes
  Controllers/
    publicas/          Catálogo, carrito, contacto y compra
    admin/             Panel, productos, pedidos y bandejas
    auth/              Acceso, solicitudes y recuperación
  Services/            Reglas de usuarios, productos y pedidos
  Views/
    publicas/          Plantillas del sitio
    admin/             Plantillas de administración
    auth/              Presentación compartida del acceso
    partials/          Cabecera, pie, tarjetas y tabla
public/                Entradas del sitio y recursos estáticos
database/              Esquema, ejemplos y migración por consola
tests/                 Pruebas de integración
docs/                  Documentación y generador de este manual
output/pdf/            Manual generado
```

Necesito cambiar...	Archivo o carpeta de partida
Lo que ocurre al enviar una pantalla	Controllers/publicas o Controllers/admin
Una regla de pedidos	Services/pedidos.php
Validación o imágenes de productos	Services/productos.php
Acceso, solicitudes o contraseñas	Services/usuarios.php
HTML de una pantalla	Views/publicas, Views/admin o Views/auth
Cabecera, pie o elementos repetidos	Views/partials
Interacciones del navegador	public/js/alas.js
Conexión a MySQL	Core/conexion.php

Los archivos pequeños que permanecen directamente en public y app/Views mantienen las direcciones anteriores. Se dejan como entradas compatibles, aunque las plantillas ahora tengan una carpeta específica. Los archivos internos tienen .htaccess para impedir su apertura directa por HTTP; PHP puede seguir incluyéndolos.

2. Cómo se procesa una petición

Ejemplo: abrir el catálogo

El navegador solicita public/productos.php. Ese archivo carga el controlador y luego la plantilla. No contiene consultas ni reglas del negocio.

```
require __DIR__ . '/../app/Controllers/publicas/productos.php';
require __DIR__ . '/../app/Views/publicas/productos.php';
```

1. El controlador carga Core/app.php una sola vez con require_once.
2. app.php establece la zona horaria, abre la sesión, conecta MySQL y carga las utilidades.
3. El controlador lee q y categoria con get(). Construye filtros permitidos y ejecuta una consulta preparada.
4. La consulta devuelve $productos. La plantilla recorre esa lista e incluye las tarjetas compartidas.
5. La cabecera incorpora public/css/alas.css y public/js/alas.js. El navegador presenta la pantalla.

Ejemplo: enviar un formulario



El servicio da uno de estos dos resultados:

Resultado correcto	Resultado con error
Guarda un mensaje y redirige a otra URL.	Conserva los valores y vuelve a mostrar el formulario.

La redirección después de guardar provoca una nueva petición GET. Así, actualizar la pantalla de resultado no repite el POST. Cada formulario de pedido tiene un nonce propio, válido durante dos horas: procesar una pestaña no invalida las otras.

3. Usuarios, permisos y recuperación

Inicio de sesión

login.php define el modo de acceso y carga auth-page.php. Esta entrada elige un controlador dentro de una lista cerrada. Controllers/auth/login.php comprueba CSRF y llama a iniciarSesion(), en Services/usuarios.php.

iniciarSesion() busca la cédula y verifica el hash con password_verify(). La cuenta debe estar aprobada. Al ingresar, regenera la sesión, guarda user_id y genera un nuevo token CSRF. Después de seis credenciales incorrectas, la cédula queda bloqueada durante un minuto, incluso en otra sesión. Los intentos se guardan en MySQL bajo bloqueo de fila.

Solicitud y aprobación

1. register.php recibe nombre, cédula, correo y dos contraseñas.
2. solicitarUsuario() valida los datos y evita duplicados de cédula o correo.
3. La cuenta se guarda como pendiente y empleado. También se registra un correo local.
4. El administrador abre accesos.php y aprueba o rechaza la solicitud.
5. revisarSolicitud() actualiza el estado y registra la respuesta en la misma transacción.

Acción	Visitante	Empleado aprobado	Administrador
Catálogo, carrito y contacto	Sí	Sí	Sí
Gestionar productos y pedidos	No	Sí	Sí
Ver consultas recibidas	No	Sí	Sí
Revisar accesos y correo local	No	No	Sí

Recuperación

solicitarRecuperacion() busca una cuenta aprobada por correo. Si existe, invalida enlaces anteriores, genera un token y guarda su hash SHA-256 en recuperaciones. El enlace vence en una hora y queda en correos_locales. La pantalla da la misma respuesta para correos conocidos y desconocidos.

restablecerContrasena() bloquea el token, comprueba su vigencia, actualiza el hash de contraseña, libera el bloqueo de intentos e invalida los enlaces de ese usuario. El token solo se puede usar una vez. La instalación guarda correo local; no tiene transporte SMTP conectado.

4. Catálogo y administración de productos

El sitio muestra únicamente productos con activo=1 y eliminado_en IS NULL. La búsqueda pública compara nombre, descripción e ingredientes. El filtro de categoría admite Panes, Bizcochos, Tortas y Especiales.

Guardado de un producto

1. **Entrada de gestión**
   agregar-producto.php o editar-producto.php define la acción.
2. **Controlador de formulario**
   Exige un usuario aprobado y conserva los campos recibidos.
3. **guardarProducto()**
   Valida el producto y procesa la imagen opcional.
4. **MySQL**
   Inserta o actualiza nombre, descripción, precio y demás campos.
5. **Listado**
   Redirección a productos.php con el resultado.

validarProducto() reúne las reglas: nombre obligatorio, categoría permitida, precio positivo con hasta dos decimales, presentación obligatoria, límites de longitud y estado activo o inactivo.

guardarImagenProducto() acepta JPG o PNG de hasta 4 MB y 8000 píxeles por dimensión. Revisa el contenido y las dimensiones en el servidor. Genera un nombre aleatorio y guarda la ruta relativa a public. Si falla el guardado en MySQL, se retira la imagen que acaba de subirse.

Imagen y eliminación

- Con imagen cargada: product_image() devuelve la URL de uploads/productos.
- Sin imagen cargada: se usa pan.svg, croissant.svg o torta.svg según la categoría.
- Inactivar oculta el producto del catálogo sin archivarlo.
- Eliminar llama a archivarProducto(): marca activo=0 y completa eliminado_en. No borra la fila ni el detalle de pedidos anteriores.

Las ilustraciones SVG siguen siendo archivos independientes en public/images. Los CSS no participan en las reglas de guardado. El estado destacado ya existente se utiliza para ordenar el catálogo; no se añadió un campo nuevo al formulario.

5. Carrito y pedido del cliente

El visitante no necesita una cuenta. El carrito se guarda en la sesión de PHP como pares de identificador de producto y cantidad. No hay una tabla de carritos.

```
$_SESSION['carrito'] = [
    12 => 2,  // Dos unidades del producto 12.
    18 => 1,  // Una unidad del producto 18.
];
```

actualizarCarrito() atiende agregar, actualizar, quitar y vaciar. Comprueba disponibilidad y cantidades entre 1 y 99. obtenerCarrito() limpia productos no disponibles al mostrar el carrito; al confirmar se usa obtenerCarrito(true), que rechaza el pedido completo si falta alguno.

Confirmación del pedido

1. **Carrito**
   Muestra productos, cantidades y total estimado.
2. **realizar-pedido.php**
   Lee nombre, teléfono, fecha, hora y notas.
3. **guardarPedido()**
   Usa los productos de la sesión y los precios de MySQL.
4. **Transacción correcta**
   Registra el pedido y todas sus líneas.
5. **Confirmación**
   Vacía el carrito, consume el nonce y muestra el número asignado.

El nonce conserva el resumen que vio el cliente. Antes de guardar se comparan productos, cantidades y precios con el carrito actual. Si cambió algo, se pide volver al carrito; no se registra un pedido parcial. El servicio vuelve a comprobar disponibilidad y calcular importes dentro de la transacción.

El identificador del último pedido confirmado queda en pedido_confirmado dentro de esa sesión. La pantalla de confirmación requiere ese dato. El pedido se crea como Pendiente y se retira en el local. La aplicación no procesa pagos en línea.

Qué ocurre si algo falla

El error vuelve al formulario y el carrito permanece disponible. No se consume el nonce de pedido hasta completar el guardado. Una petición sin un CSRF válido también se rechaza antes de modificar datos.

6. El servicio de pedidos, paso por paso

Tanto la compra pública como el formulario del equipo llaman a guardarPedido(), en Services/pedidos.php. Este punto común evita mantener dos implementaciones distintas del cálculo y del guardado.

Paso	Función	Responsabilidad
1	validarDatosPedido	Cliente, teléfono, retiro, notas y estado
2	cargarHistorialPedido	Bloquear, comprobar versión y leer el historial
3	prepararLineaPedido	Validar cantidad y decidir nombre y precio
4	calcularTotalPedido	Sumar en centavos y controlar el máximo
5	guardarCabeceraPedido	Insertar o actualizar los datos generales
6	guardarPedido	Guardar detalles y confirmar la transacción

La transacción

begin_transaction() inicia la unidad de trabajo. Los SELECT FOR UPDATE protegen los registros mientras se guardan cambios. commit() confirma la cabecera y las líneas juntas. Cualquier excepción ejecuta rollback() y conserva el pedido anterior.

El formulario envía pedido_version. obtenerPedidoEditable() la compara con pedidos.version bajo bloqueo. Si otra persona guardó cambios, se rechaza la edición y se pide reabrir el pedido. Cada guardado o cancelación incrementa la versión. guardarPedido() y cancelarPedido() requieren esa versión al modificar un pedido existente.

Por qué se conserva el historial

Ejemplo: un producto cuesta $ 10,25 cuando se registra un pedido. Si luego pasa a costar $ 88, editar la cantidad de ese mismo producto mantiene $ 10,25 como precio unitario. También se conserva el nombre original. Un producto nuevo que se agrega a ese pedido usa el precio actual.

Los detalles importados pueden tener id_producto vacío. La opción historico:ID_DETALLE permite mantenerlos y editar su cantidad. Solo se acepta si ese detalle pertenece al pedido que se está editando.

Reglas que no se deben perder

- Entre 1 y 50 líneas; cantidad entera entre 1 y 9999 por línea.
- Un pedido nuevo requiere un retiro futuro. En una edición se admiten fechas anteriores.
- Los productos inactivos o archivados solo se conservan si ya estaban en ese pedido.
- Entregado y Cancelado son estados de solo lectura.
- cancelarPedido() exige un motivo permitido y conserva cabecera y detalles.
- El resumen producto y la cantidad total se mantienen por compatibilidad con los registros y listados anteriores.

7. Cómo se organiza la base de datos

La base se llama alas_bd. estructura.sql describe una instalación nueva y migrar.php actualiza las anteriores. migrar-correcciones.php agrega únicamente pedidos.version e intentos_acceso a una instalación completa, sin cargar datos demo.

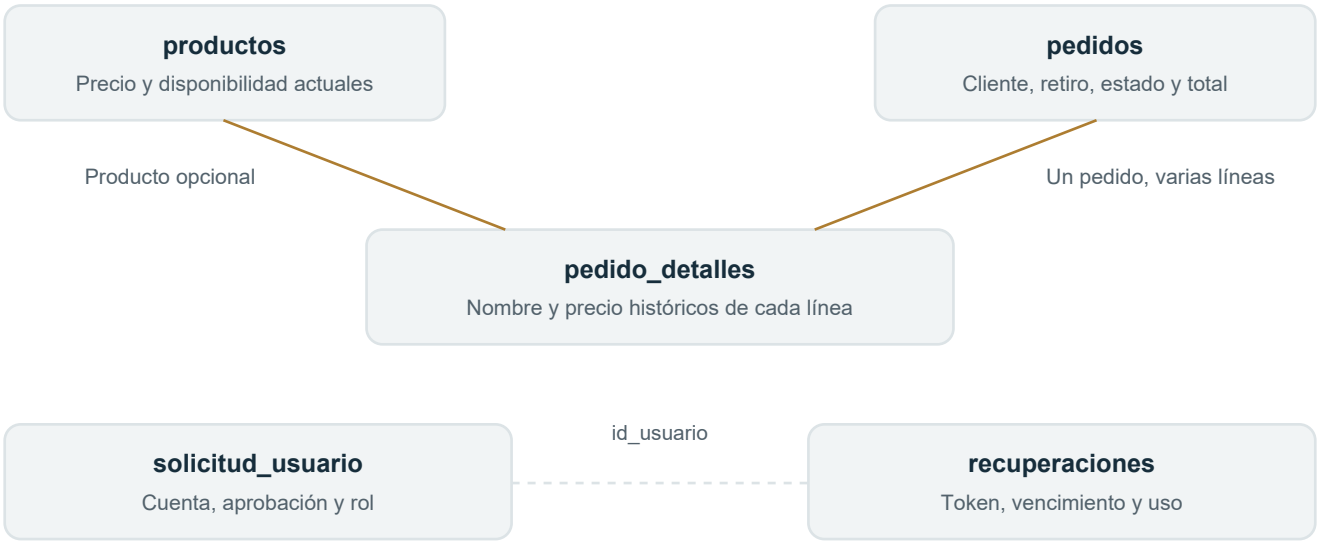


Tabla	Qué guarda y cómo se relaciona
solicitud_usuario	Cédula, hash de contraseña, nombre, correo, estado y rol. id_sol identifica al usuario.
productos	Catálogo, precio actual, categoría, ingredientes, imagen y archivo lógico.
pedidos	Cliente, retiro, estado, total, versión, cancelación y resumen de productos.
pedido_detalles	Líneas del pedido: producto opcional, nombre histórico, cantidad y precio histórico.
mensajes_contacto	Consultas del sitio y marca de leído.
recuperaciones	Usuario, hash del token, vencimiento y marca de usado.
correos_locales	Destinatario, asunto y cuerpo de mensajes de desarrollo.
intentos_acceso	Intentos fallidos y vencimiento del bloqueo por cédula, independiente de la sesión.

pedido_detalles tiene claves foráneas hacia pedidos y productos. Eliminar físicamente un pedido borra sus detalles por la relación; eliminar físicamente un producto deja la referencia del detalle en NULL. El funcionamiento normal archiva productos y cancela pedidos, conservando los registros.

recuperaciones.id_usuario se utiliza para buscar solicitud_usuario.id_sol; es una relación lógica en el esquema actual. El carrito y los tokens de formulario pertenecen a la sesión, no a estas tablas. La tabla antigua usuarios, si está presente, no participa en el inicio de sesión actual.

8. Entradas que se abren en el navegador

Sitio público

Ruta desde la raíz del proyecto	Función
public/index.php o public/inicio.php	Inicio y productos favoritos
public/productos.php	Catálogo, búsqueda y categorías
public/producto.php?id=...	Detalle y agregado al carrito
public/nosotros.php	Presentación de la panadería
public/contacto.php	Formulario de consultas
public/mensaje-enviado.php	Confirmación de una consulta de la sesión
public/carrito.php	Cambiar productos y cantidades
public/realizar-pedido.php	Datos del cliente y confirmación
public/pedido-confirmado.php	Número de pedido de la sesión

Acceso y administración

Ruta dentro de app/Views	Función
login.php, register.php	Iniciar sesión o solicitar acceso
recuperar.php, restablecer.php	Recuperación y nueva contraseña
solicitud-enviada.php	Confirmar registro o recuperación
principal.php o panel.php	Cifras reales y próximos retiros
productos.php y pedidos.php	Listados, búsquedas y filtros
agregar/editar/eliminar-producto.php	Crear, editar o archivar un producto
agregar/editar/eliminar-pedido.php	Crear, editar o cancelar un pedido
accesos.php, consultas.php, correos.php	Aprobaciones y bandejas

Los nombres con agregar/editar/eliminar representan los tres archivos existentes de cada entidad. gestion.php elige el formulario correspondiente mediante una lista cerrada. La salida usa POST a app/Controllers/logout.php y requiere CSRF.

El listado de pedidos muestra 20 por página. Los paneles de solicitudes y bandejas muestran hasta 100 registros; próximos retiros muestra hasta cinco. Los enlaces y parámetros q, estado, fecha, pagina e id se conservan.

9. Utilidades compartidas y JavaScript

Funciones pequeñas de Core

Archivo	Funciones y uso
app.php	Carga común de sesión, conexión y utilidades
base-datos.php	sql(): ejecuta; rows(): lista; one(): fila o null
http.php	get(), post(), base_url(), redirect() y flash()
seguridad.php	CSRF, nonces por formulario, current_user() y require_user()
presentacion.php	e(), money(), badge(), imágenes y show_flash()
validacion.php	validarCorreo(), validate_text() y error_text()
correo.php	queue_mail(): inserta en la bandeja local
auth.php	Exige sesión aprobada y define $usuarioActual

sql() usa consultas preparadas. La cadena de tipos indica s para texto, i para entero y d para decimal. Los valores se pasan por separado, no se concatenan dentro del SQL.

```
sql('UPDATE productos SET nombre=? WHERE id_producto=?',
    'si', [$nombre, $idProducto]);
```

e() escapa contenido antes de mostrarlo en HTML. error_text() permite mostrar errores de validación y registra los fallos internos sin exponer la consulta o la excepción al visitante. Core/pedidos.php conserva las funciones antiguas save_order() y order_lines() como compatibilidad; el código nuevo usa Services/pedidos.php.

Qué hace public/js/alas.js

- iniciarBotonesContrasena(): alterna mostrar y ocultar la contraseña.
- iniciarVistaPreviaProducto(): revisa tipo y tamaño, muestra la imagen elegida y cambia el SVG por categoría cuando corresponde.
- iniciarFormularioPedido(): conecta los eventos del formulario con las funciones de líneas y resumen.
- agregarLineaPedido(): copia una línea vacía hasta un máximo de 50.
- actualizarResumenPedido(): muestra productos, cantidades, total y retiro.

Los atributos data-* vinculan el HTML con JavaScript. Cambiar esos nombres exige actualizar ambos archivos. El servidor mantiene las validaciones y el cálculo definitivo aunque JavaScript esté desactivado. public/js/auth.js es un archivo anterior; la cabecera actual carga alas.js.

10. Cómo modificar una funcionalidad

Ejemplo: agregar un dato opcional de producto

1. Definir la columna y su valor inicial en database/estructura.sql. Preparar una actualización para las bases existentes sin borrar datos.
2. Agregar el campo a los valores del controlador admin/producto-form.php y a la lista de campos editables que lee del POST.
3. Incorporar su validación en Services/productos.php. Agregar el parámetro en INSERT y UPDATE y mantener la cadena de tipos alineada.
4. Mostrar el control en Views/admin/producto-form.php. Usar e() al imprimir su valor.
5. Si es público, mostrarlo también en Views/publicas/producto.php o en la tarjeta que corresponda.
6. Probar alta, edición, valor vacío y datos inválidos. Revisar también los productos anteriores que todavía no tenían ese dato.

Ejemplo: ajustar una regla de pedidos

La regla debe quedar en Services/pedidos.php para que afecte al pedido del cliente y al pedido interno. Si solo cambia el resumen visual, se modifica alas.js. Si cambia un estado, revisar states(), los filtros y las comprobaciones de solo lectura y cancelación.

Mantener el código simple

- El controlador decide qué hacer con la petición y qué pantalla mostrar.
- El servicio aplica una operación del negocio y devuelve un resultado o lanza una excepción.
- La vista recorre datos y produce HTML. No ejecuta consultas SQL ni guarda formularios.
- Core contiene utilidades usadas por varias funcionalidades.
- Los nombres de variables y las funciones del negocio describen su intención en español.
- Los comentarios explican una decisión, como conservar un precio histórico, en lugar de repetir cada instrucción.

No hace falta incorporar un framework para seguir este esquema. Se conserva mysqli, la conexión compartida y los includes explícitos para que el equipo pueda seguir el recorrido desde el archivo de entrada.

11. Arranque y verificación

Ejecutar en XAMPP

Iniciar Apache y MySQL. Para una instalación nueva, importar estructura.sql y luego datos_demo.sql si se necesitan ejemplos. Ejecutar la migración desde la raíz del proyecto. En una base anterior, ejecutar primero la migración antes de importar los ejemplos. Revisar Core/conexion.php si el servidor utiliza otra base o credenciales.

```
C:\xampp\php\php.exe database\migrar.php
```

Abrir http://localhost/software-pedidos-alas/public/index.php. Los datos de la cuenta local de demostración están en README.md. La migración no se ejecuta al visitar el sitio.

Pruebas disponibles

```
# PowerShell 7, Apache y MySQL activos; datos demo cargados.
pwsh -NoProfile -File tests\integracion.ps1

# También comprueba la carga de una imagen temporal.
pwsh -NoProfile -File tests\integracion.ps1 -ProbarImagen

# Reglas y conservación del historial contra MySQL.
C:\xampp\php\php.exe tests\pedidos.php
C:\xampp\php\php.exe tests\catalogo.php
node --test tests\imagenes.test.js

# Sintaxis de las interacciones del navegador.
node --check public\js\alas.js
```

integracion.ps1 admite BaseUrl, Mysql, CedulaAdministrador y ContrasenaAdministrador. Utiliza una base local de desarrollo con ejemplos; crea registros con una marca aleatoria y los retira en finally. La opción de imagen requiere permiso para escribir y retirar archivos en public/uploads/productos.

Verificación	Qué cubre
Integración HTTP	Acceso, panel, catálogo, productos, pedidos, carrito y contacto
Identidad y permisos	Pendientes, aprobación, recuperación de un uso y acceso privado
Historial	Precios, versiones, productos archivados y detalles antiguos
Rechazos	CSRF, teléfono, importes, edición anterior y carrito incompleto
Navegador	Líneas dinámicas, resumen e imágenes opcionales

Los ocho archivos CSS se compararon por SHA-256 y permanecieron iguales respecto del inicio de esta reorganización. Las pruebas revisan los casos indicados; no sustituyen una prueba de carga o una revisión para publicar en producción.

12. Diagnóstico y lectura rápida

Síntoma	Dónde empezar a revisar
No conecta con MySQL	Servicio MySQL, alas_bd y Core/conexion.php
No permite iniciar sesión	solicitud_usuario, estado aprobado y hash de contraseña
Formulario informa sesión vencida	Campo CSRF, cookie de sesión y check_csrf()
No aparece un producto	activo, eliminado_en y filtros del catálogo
El pedido no se puede editar	Estados Entregado o Cancelado
Pedido modificado o formulario anterior	Volver al listado y reabrir; otra edición avanzó su versión
El precio anterior no cambia	Es la conservación del historial del pedido
No llega un correo	Consultar correos.php; no hay envío SMTP conectado
No cambia el resumen al elegir	Carga de alas.js y atributos data-* del formulario
Error 403 en una carpeta interna	Abrir la ruta de entrada, no la plantilla o el servicio

Glosario práctico

- Sesión: datos conservados por PHP para ese navegador, como usuario o carrito.
- CSRF: token que el formulario debe devolver para autorizar una modificación desde esa sesión.
- Nonce del pedido: valor de un solo uso por formulario; vence a las dos horas sin afectar otras pestañas.
- Transacción: grupo de escrituras que se confirma entero o se revierte entero.
- Eliminación lógica: ocultar o archivar un registro conservando sus datos.
- Snapshot histórico: nombre y precio guardados en el detalle de un pedido, independientes del catálogo actual.
- Plantilla: archivo que produce HTML a partir de variables preparadas por el controlador.

Recorrido recomendado para una persona nueva

1. Abrir public/productos.php y seguir sus dos require.
2. Leer Controllers/publicas/productos.php y después Views/publicas/productos.php.
3. Seguir realizar-pedido.php hasta guardarPedido() para entender las escrituras.
4. Leer el servicio de usuarios y el guardado de productos.
5. Ejecutar las pruebas y consultar este manual al modificar un flujo.

La fuente editable de esta guía está en docs/guia-codigo.md. docs/generar_manual.py produce el PDF con ReportLab. El contenido describe la implementación local reorganizada, incluidas sus limitaciones actuales de correo y pago.